TECHNICAL PROJECT GUIDE

# Godot 4 Temple Runner

Setup, Gameplay, and 3D Environment Development

Windows 11 • Godot 4 • Blender • GDScript

## Project brief

A practical guide to developing a 3D endless runner set on an ancient temple path, with jungle scenery, obstacles, collectible orbs, and colourful gates.

### Scope of this guide

- Configure a Godot 4 project on Windows 11.
- Implement lane movement, jumping, scoring, obstacles, and restart logic.
- Model low-poly trees and colourful gates in Blender.
- Import `.glb` assets into Godot and recycle scenery during gameplay.
- Prepare the project for future colour-theory mechanics and document the AI prompt-response history.

**Prepared by : Sk Zaid**

**SAP ID : 590022608**

### GitHub Repository

`github.com/skzaid712601-eng/Colour-Theory-Temple-Runner`

# Contents

**Executive Summary** **2**

**1 Project Overview** **3**
1.1 Prototype features . . . 3

**2 Install and Prepare Godot on Windows 11** **3**

**3 Current Game Script** **4**
3.1 Run the prototype . . . 9

**4 Create a Low-Poly Jungle Tree in Blender** **10**
4.1 Build the trunk . . . 10
4.2 Add foliage and materials . . . 10
4.3 Export the tree . . . 10

**5 Create a Colourful Gate in Blender** **10**
5.1 Model the frame . . . 10
5.2 Apply colours and export . . . 10

**6 Import Blender Models into Godot** **11**
6.1 Add model references . . . 11
6.2 Create and position the scenery . . . 11
6.3 Move and recycle trees and gates . . . 12

**7 Testing Checklist** **12**

**8 Troubleshooting** **12**

**9 Suggested Next Development Stage** **13**

**A AI Prompt and Response History** **14**
A.1 Interaction 1: Godot 4 Installation . . . 14
A.2 Interaction 2: Building the Temple Runner . . . 16
A.3 Interaction 3: Boolean Type-Inference Error . . . 23
A.4 Interaction 4: Indentation Error . . . 24
A.5 Interaction 5: Request for Corrected Full Script . . . 25
A.6 Interaction 6: Adding Blender Scenery . . . 33

## Executive Summary

This guide documents a beginner-scale Temple Runner prototype made with Godot 4 and GDScript, along with a workflow for creating and importing custom environment assets from Blender. It covers the initial game script, the essential controls, low-poly tree and gate modelling, asset import, scenery recycling, and a testing checklist. The colourful gates described here are scenery only; collision-based gate challenges and colour-theory rules are listed as future development work. An appendix reproduces the AI prompts and responses supplied for the project.

**Implementation note**

The document is intended as a practical starting point rather than a claim that the prototype is production-ready. Test the script and imported assets in the exact Godot and Blender versions installed on your computer before submitting the project.

# 1 Project Overview

Godot 4 is a free, open-source game engine for developing 2D and 3D games. This project uses Godot's 3D nodes and GDScript to create a simple temple-running prototype. Blender is optional, but useful for creating custom scenery.

## 1.1 Prototype features

- Three-lane movement using the left/right arrow keys or `A`/`D`.
- Jumping using `Space` or `Enter`.
- Collectible cyan orbs that add points.
- Obstacles that end the run if the player collides with them while grounded.
- Increasing running speed and a distance counter.
- A game-over message and restart control.

# 2 Install and Prepare Godot on Windows 11

1. Visit the official download page: https://godotengine.org/download/windows/.
2. For most Windows 11 computers, download the standard **Windows x86_64** version. Choose the .NET version only if you specifically need C# support.
3. Extract the downloaded ZIP archive using File Explorer.
4. Run the Godot executable from the extracted folder.
5. In Project Manager, create a project named `TempleRunner3D`.
6. Create a 3D scene, rename its root node to `Main`, and save it as `main.tscn`.
7. Attach a script named `main.gd` to the `Main` node.

**Practical tip**

Godot can run without a traditional installer. If your computer has graphics compatibility problems, try the Compatibility renderer in the project settings.

## 3 Current Game Script

The following is a complete, single-file beginner prototype. Replace the contents of `main.gd` with the code below. The obstacle proximity variables are explicitly typed as `bool`, and the collision check is kept inside the obstacle loop to avoid the earlier parser and indentation errors.

```
extends Node3D

# TEMPLE RUNNER - BEGINNER VERSION
# Controls: A/D or arrows, Space/Enter to jump, R to restart.

var player: CharacterBody3D
var camera: Camera3D
var score_label: Label
var message_label: Label

var obstacles: Array[Node3D] = []
var orbs: Array[Node3D] = []
var pillars: Array[Node3D] = []

var lane: int = 1
var lanes: Array[float] = [-2.0, 0.0, 2.0]

var speed: float = 12.0
var gravity: float = 24.0
var jump_power: float = 9.0

var score: int = 0
var distance: float = 0.0
var spawn_timer: float = 0.0
var game_over: bool = false


func _ready() -> void:
    randomize()
    make_world()
    make_player()
    make_camera()
    make_ui()
    spawn_timer = 1.0


func make_material(color: Color, glow: bool = false) -> StandardMaterial3D:
    var mat := StandardMaterial3D.new()
    mat.albedo_color = color
    if glow:
        mat.emission_enabled = true
        mat.emission = color
        mat.emission_energy_multiplier = 2.0
    return mat


func make_box(
    parent: Node3D,
    size: Vector3,
    pos: Vector3,
    color: Color
) -> MeshInstance3D:
    var mesh_instance := MeshInstance3D.new()
    var mesh := BoxMesh.new()
    mesh.size = size
    mesh_instance.mesh = mesh
    mesh_instance.position = pos
```

```
    mesh_instance.material_override = make_material(color)
    parent.add_child(mesh_instance)
    return mesh_instance


func make_world() -> void:
    # Jungle-temple atmosphere.
    var environment_node := WorldEnvironment.new()
    var environment := Environment.new()
    environment.background_mode = Environment.BG_SKY
    environment.ambient_light_source = Environment.AMBIENT_SOURCE_COLOR
    environment.ambient_light_color = Color(0.55, 0.65, 0.52)
    environment.ambient_light_energy = 0.8
    environment_node.environment = environment
    add_child(environment_node)

    var sun := DirectionalLight3D.new()
    sun.rotation_degrees = Vector3(-45, -25, 0)
    sun.light_energy = 1.3
    add_child(sun)

    # Main stone pathway.
    var floor_body := StaticBody3D.new()
    floor_body.name = "TemplePath"
    add_child(floor_body)

    make_box(
        floor_body,
        Vector3(7, 0.5, 120),
        Vector3(0, -0.25, -25),
        Color(0.36, 0.32, 0.26)
    )

    var floor_collision := CollisionShape3D.new()
    var floor_shape := BoxShape3D.new()
    floor_shape.size = Vector3(7, 0.5, 120)
    floor_collision.shape = floor_shape
    floor_collision.position = Vector3(0, -0.25, -25)
    floor_body.add_child(floor_collision)

    # Decorative path markings.
    for z in range(-80, 20, 5):
        make_box(
            self,
            Vector3(0.06, 0.02, 2),
            Vector3(0, 0.015, float(z)),
            Color(0.58, 0.49, 0.34)
        )

    # Temple pillars.
    for z in range(-55, 5, 8):
        make_pillar(-4.3, float(z))
        make_pillar(4.3, float(z))


func make_pillar(x: float, z: float) -> void:
    var pillar := Node3D.new()
    pillar.position = Vector3(x, 0, z)
    add_child(pillar)

    make_box(
        pillar,
        Vector3(0.8, 4, 0.8),
        Vector3(0, 2, 0),
```

```
        Color(0.48, 0.43, 0.34)
    )

    make_box(
        pillar,
        Vector3(1.2, 0.35, 1.2),
        Vector3(0, 4, 0),
        Color(0.64, 0.53, 0.35)
    )

    pillars.append(pillar)


func make_player() -> void:
    player = CharacterBody3D.new()
    player.name = "Player"
    player.position = Vector3(0, 0, 0)
    add_child(player)

    var capsule := CapsuleMesh.new()
    capsule.radius = 0.4
    capsule.height = 1.8

    var body_mesh := MeshInstance3D.new()
    body_mesh.mesh = capsule
    body_mesh.position.y = 0.9
    body_mesh.material_override = make_material(Color(0.15, 0.65, 1.0))
    player.add_child(body_mesh)

    var collision := CollisionShape3D.new()
    var shape := CapsuleShape3D.new()
    shape.radius = 0.4
    shape.height = 1.8
    collision.shape = shape
    collision.position.y = 0.9
    player.add_child(collision)


func make_camera() -> void:
    camera = Camera3D.new()
    camera.position = Vector3(0, 5, 8)
    add_child(camera)
    camera.current = true
    camera.fov = 75
    camera.look_at(Vector3(0, 1, -8), Vector3.UP)


func make_ui() -> void:
    var canvas := CanvasLayer.new()
    add_child(canvas)

    score_label = Label.new()
    score_label.position = Vector2(20, 20)
    score_label.add_theme_font_size_override("font_size", 26)
    score_label.add_theme_color_override("font_color", Color(1.0, 0.85, 0.3))
    canvas.add_child(score_label)

    message_label = Label.new()
    message_label.anchor_left = 0.15
    message_label.anchor_top = 0.35
    message_label.anchor_right = 0.85
    message_label.anchor_bottom = 0.65
    message_label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    message_label.vertical_alignment = VERTICAL_ALIGNMENT_CENTER
```

```
    message_label.add_theme_font_size_override("font_size", 30)
    message_label.add_theme_color_override("font_color", Color.WHITE)
    canvas.add_child(message_label)

    message_label.text = "TEMPLE RUNNER\nA/D or arrows to move\nSPACE or ENTER to jump"


func spawn_obstacle() -> void:
    var obstacle := Area3D.new()
    obstacle.name = "StoneObstacle"

    var obstacle_mesh := BoxMesh.new()
    obstacle_mesh.size = Vector3(1.2, 1.2, 1.2)

    var visual := MeshInstance3D.new()
    visual.mesh = obstacle_mesh
    visual.position.y = 0.6
    visual.material_override = make_material(Color(0.65, 0.22, 0.12))
    obstacle.add_child(visual)

    var collision := CollisionShape3D.new()
    var shape := BoxShape3D.new()
    shape.size = Vector3(1.2, 1.2, 1.2)
    collision.shape = shape
    collision.position.y = 0.6
    obstacle.add_child(collision)

    var chosen_lane: int = randi_range(0, 2)
    obstacle.position = Vector3(lanes[chosen_lane], 0, -40)
    add_child(obstacle)
    obstacles.append(obstacle)


func spawn_orb() -> void:
    var orb := Area3D.new()
    orb.name = "GoldenOrb"

    var orb_mesh := SphereMesh.new()
    orb_mesh.radius = 0.38
    orb_mesh.height = 0.76

    var visual := MeshInstance3D.new()
    visual.mesh = orb_mesh
    visual.material_override = make_material(Color(0.1, 0.95, 1.0), true)
    orb.add_child(visual)

    var collision := CollisionShape3D.new()
    var shape := SphereShape3D.new()
    shape.radius = 0.65
    collision.shape = shape
    orb.add_child(collision)

    var chosen_lane: int = randi_range(0, 2)
    orb.position = Vector3(lanes[chosen_lane], 1.1, -40)
    add_child(orb)
    orbs.append(orb)


func _input(event: InputEvent) -> void:
    if game_over:
        if event.is_action_pressed("ui_accept") \
        or event.is_action_pressed("ui_cancel") \
        or (event is InputEventKey and event.pressed and event.keycode == KEY_R):
            get_tree().reload_current_scene()
```

```
        return

    # Support both the default arrow actions and A/D keyboard keys.
    var move_left: bool = event.is_action_pressed("ui_left")
    var move_right: bool = event.is_action_pressed("ui_right")

    if event is InputEventKey and event.pressed and not event.echo:
        move_left = move_left or event.keycode == KEY_A
        move_right = move_right or event.keycode == KEY_D

    if move_left:
        lane = maxi(0, lane - 1)

    if move_right:
        lane = mini(2, lane + 1)

    if event.is_action_pressed("ui_accept"):
        if player.is_on_floor():
            player.velocity.y = jump_power


func _physics_process(delta: float) -> void:
    if game_over:
        return

    # Smoothly move between lanes.
    player.position.x = move_toward(
        player.position.x, lanes[lane], 10.0 * delta
    )

    # Jumping and gravity.
    if not player.is_on_floor():
        player.velocity.y -= gravity * delta
    elif player.velocity.y < 0:
        player.velocity.y = 0

    player.move_and_slide()

    # Increase speed and distance.
    speed = min(speed + 0.12 * delta, 24.0)
    distance += speed * delta
    spawn_timer -= delta

    if spawn_timer <= 0:
        spawn_obstacle()
        if randf() < 0.8:
            spawn_orb()
        spawn_timer = randf_range(1.0, 1.6)

    # Move obstacles towards the player.
    for obstacle in obstacles.duplicate():
        obstacle.position.z += speed * delta

        if obstacle.position.z > 3:
            obstacles.erase(obstacle)
            obstacle.queue_free()
            continue

        var close_x: bool = abs(
            obstacle.position.x - player.position.x
        ) < 0.85

        var close_z: bool = abs(
            obstacle.position.z - player.position.z
```

```
        ) < 0.8

        # Keep collision detection inside the obstacle loop.
        if close_x and close_z and player.position.y < 0.85:
            end_game()

    # Move collectible orbs.
    for orb in orbs.duplicate():
        orb.position.z += speed * delta
        orb.rotate_y(3.0 * delta)

        if orb.position.z > 3:
            orbs.erase(orb)
            orb.queue_free()
            continue

        if orb.global_position.distance_to(
            player.global_position + Vector3(0, 1, 0)
        ) < 1.0:
            score += 10
            orbs.erase(orb)
            orb.queue_free()

    # Move decorative pillars.
    for pillar in pillars:
        pillar.position.z += speed * delta
        if pillar.position.z > 10:
            pillar.position.z -= 64

    score_label.text = (
        "ORBS: " + str(score)
        + "     DISTANCE: " + str(int(distance))
    )


func end_game() -> void:
    game_over = true
    message_label.text = (
        "GAME OVER!\n"
        + "Orbs: " + str(score)
        + "\nDistance: " + str(int(distance))
        + "\nPress R to restart"
    )
```

**Listing 1:** main.gd — Temple Runner prototype

## 3.1 Run the prototype

1. Save `main.gd` with `Ctrl+S`.
2. Press `F6` to run the current scene, or `F5` to run the project.
3. If asked for a main scene, select `main.tscn`.
4. Click inside the game window before using the controls.

| Key | Action |
|---|---|
| Left arrow / A | Move left one lane |
| Right arrow / D | Move right one lane |
| Space / Enter | Jump |
| R | Restart after game over |

# 4 Create a Low-Poly Jungle Tree in Blender

The goal is a lightweight stylized tree with a brown trunk and several green foliage clusters.

## 4.1 Build the trunk

1. Open Blender and create a new General project.
2. Select the default cube and delete it.
3. Use `Shift+A` and choose `Mesh > Cylinder`.
4. Scale and position the cylinder to form a tall, narrow trunk. For example, scale along Z, narrow the horizontal dimensions, and move it upward so its base sits at the origin.

## 4.2 Add foliage and materials

1. Add an `Icosphere` using `Shift+A > Mesh > Icosphere`.
2. Move it above the trunk and scale it to form a canopy.
3. Duplicate the foliage twice to create a fuller silhouette; vary the size and height.
4. Give the trunk a brown material. Give the foliage several green shades.
5. Use flat shading for a simple low-poly appearance.

## 4.3 Export the tree

1. Select the trunk and foliage objects.
2. Join them if desired using `Ctrl+J`; check that the material assignments remain correct.
3. Name the model `JungleTree`.
4. Save the Blender project as `JungleTree.blend`.
5. Choose `File > Export > glTF 2.0` and export as **glTF Binary (.glb)** named `JungleTree.glb`.

# 5 Create a Colourful Gate in Blender

Create a rectangular frame that can later be used as scenery or developed into a colour-theory challenge.

## 5.1 Model the frame

1. Start a new Blender General project and delete the default cube.
2. Add a cube for one vertical post. Scale it narrow in X and Y and tall in Z.
3. Duplicate it along X to make the second post.
4. Add another cube, scale it into a horizontal beam, and position it across the top.
5. Keep the frame approximately 3–4 metres wide and around 3 metres tall, adjusting the dimensions to fit your path.

## 5.2 Apply colours and export

Use three clearly distinguishable materials:

- Red: `#FF3030`
- Blue: `#168BFF`
- Green: `#36FF75`

Assign materials to different sections of the frame. Emission can help the colours appear bright, although the final appearance may differ after import.

Name the object `ColourGate`, save the Blender project as `ColourGate.blend`, and export it as `ColourGate.glb` through `File > Export > glTF 2.0 > glTF Binary (.glb)`.

# 6 Import Blender Models into Godot

1. In the Godot FileSystem dock, create a folder under `res://` named `assets`.
2. Copy `JungleTree.glb` and `ColourGate.glb` into `assets`.
3. Wait for Godot to import the files.
4. If necessary, drag each model into a temporary 3D scene to check its scale, orientation, and materials.

## 6.1 Add model references

Add these declarations near the top of `main.gd`, below the existing arrays. Keep the filenames and capitalization exactly as shown.

```
var trees: Array[Node3D] = []
var gates: Array[Node3D] = []

const TREE_SCENE: PackedScene = preload(
    "res://assets/JungleTree.glb"
)
const GATE_SCENE: PackedScene = preload(
    "res://assets/ColourGate.glb"
)
```

## 6.2 Create and position the scenery

Add these functions outside the other functions in `main.gd`.

```
func make_trees() -> void:
    for i in range(12):
        for side in [-1.0, 1.0]:
            var tree := TREE_SCENE.instantiate() as Node3D
            add_child(tree)
            tree.position = Vector3(
                side * randf_range(5.0, 7.0),
                0.0,
                -float(i) * 10.0 - 10.0
            )
            tree.scale = Vector3.ONE * randf_range(0.8, 1.2)
            trees.append(tree)


func make_gates() -> void:
    for i in range(3):
        var gate := GATE_SCENE.instantiate() as Node3D
        add_child(gate)
        gate.position = Vector3(
            0.0,
            0.0,
            -40.0 - float(i) * 45.0
        )
        gates.append(gate)
```

Update `_ready()` so the scenery is created before the player and UI:

```
func _ready() -> void:
    randomize()
    make_world()
    make_trees()
    make_gates()
    make_player()
```

```
    make_camera()
    make_ui()
    spawn_timer = 1.0
```

### 6.3 Move and recycle trees and gates

At the end of `_physics_process(delta)`, after the existing pillar loop, add the following code. It must remain inside the function.

```
    # Move trees towards the player and recycle them.
    for tree in trees:
        tree.position.z += speed * delta
        if tree.position.z > 10.0:
            tree.position.z -= 120.0

    # Move gates towards the player and recycle them.
    for gate in gates:
        gate.position.z += speed * delta
        if gate.position.z > 10.0:
            gate.position.z -= 135.0
```

**Important:** The gates in this version are decorative only. They do not yet have collision or colour-dependent gameplay effects. The model's origin and scale can affect how it appears in the scene; adjust the Blender model or its Godot transform if it is too large, too small, or positioned incorrectly.

## 7 Testing Checklist

Use this checklist after importing the models.

- ☐ The game starts without script errors.
- ☐ The stone pathway and temple pillars are visible.
- ☐ The player can move between three lanes.
- ☐ The player can jump.
- ☐ Collecting cyan orbs increases the score by 10.
- ☐ Obstacles end the run when hit while the player is grounded.
- ☐ Trees appear on both sides of the path.
- ☐ Red, blue, and green gates appear along the route.
- ☐ Trees and gates recycle as they pass the player.

## 8 Troubleshooting

| Problem | What to check |
|---|---|
| Parser or indentation error | Make sure each function body is indented consistently. The obstacle collision test belongs inside `for obstacle in obstacles.duplicate()`. |
| Cannot infer type for `close_x` or `close_z` | Declare both as `var close_x: bool = ...` and `var close_z: bool = ...`. |
| Black screen or wrong scene | Confirm `main.tscn` is selected as the main scene and the script is attached to the root `Main` node. |

| | |
|---|---|
| Player falls through floor | Confirm the floor's `StaticBody3D` has a correctly sized `CollisionShape3D`. |
| Model does not appear | Check the path under `res://assets/`, spelling, capitalization, import completion, model scale, and position. |
| Model looks too large or too small | Adjust its scale in Blender or the `tree.scale` value in Godot. |

## 9 Suggested Next Development Stage

Once the scenery is visible and stable, develop the colour-theory mechanics:

1. Give each gate a clearly labelled or visually distinct colour.
2. Decide what the player must do to pass safely through a gate.
3. Add collision or trigger detection for gates.
4. Connect the gate behaviour to colour concepts such as complementary colours, warm and cool colours, or RGB mixing.
5. Add sound effects, a start menu, improved lighting, and a proper animated character when the basic gameplay is reliable.

## Official Resources

- Godot Engine download: https://godotengine.org/download/windows/
- Godot documentation: https://docs.godotengine.org/
- Blender: https://www.blender.org/

# A AI Prompt and Response History

This appendix records the prompts and AI responses supplied for the development of the *Temple Runner* project. The exchanges are retained in chronological order to document how the project progressed from setting up Godot, to implementing the core gameplay, resolving script errors, and adding Blender-created scenery. Code and technical details are preserved in the transcript format.

**Transcript note.** The conversation has been organized into prompt-and-response pairs. Interface timing annotations are omitted; the user prompts and substantive response content are included below.

## A.1 Interaction 1: Godot 4 Installation

### User prompt

```
what is godot 4 and how to download it in windows 11
```

### AI response

```
What is Godot 4?

Godot 4 is a free, open-source game development engine that lets you create 2D and 3D games. You can use it
  ↪ to build games
for Windows, Android, Linux, and the web.
For example, you can create a 3D temple runner, a colour theory quiz game, a platformer, or a racing game
  ↪ using Godot.
How to download Godot 4 on Windows 11

Step 1: Open the official website
Click the link below.
Download Godot 4 for Windows
The official download page currently lists Godot 4.7.2 as the stable release.

Godot Engine

Step 2: Choose the correct version
For most Windows 11 computers, choose:
-   Godot Engine --- Windows x86_64
-   Choose the standard version, not the .NET version.
The standard version supports GDScript, Godot's beginner-friendly scripting language.

Step 3: Extract the downloaded file
1.  Open your Downloads folder.
2.  Right-click the downloaded ZIP file.
3.  Select Extract All.
4.  Click Extract.

Step 4: Launch Godot
1.  Open the extracted folder.
2.  Double-click the Godot executable.
3.  In the Project Manager, click Create or Create New Project.
4.  Enter a project name, choose a folder, and create your project.
Good to know: Godot does not require a traditional installation. You can extract it and run it directly.

Godot Engine
What can you make with Godot 4?

2D games
Platformers, puzzle games, quizzes, and arcade games.

3D games
Temple runners, obstacle courses, adventure games, and exploration games.

Coding and game logic
Use GDScript, a language with Python-like syntax, to control players, enemies, scoring, physics, and menus.
What do you need before starting?
-   Windows 11 PC or laptop.
-   A graphics processor supporting at least OpenGL 3.3; Vulkan-compatible hardware is recommended for
  ↪ Godot 4.
```

```
Godot Engine
-   No paid licence required.
-   Blender is optional if you want to create your own 3D models.
My recommendation: Start with the standard Godot 4 version and learn the basics before moving to a full 3D
  ↪ game.
Would you like to build your first game in Godot?
Choose your first project
3D Temple Runner --- obstacles and collectible orbs
2D Platformer --- jumping and enemies
Colour Theory Quiz --- questions and scoring
Your coding experience
Beginner --- explain every step
I know some programming
```

## A.2 Interaction 2: Building the Temple Runner

### User prompt

```
Help me create a 3D Temple Runner game in Godot 4 on Windows 11. My experience: Beginner. Give me step-by-
  ↪ step instructions and working code.
```

### AI response

```
Let's build a 3D Temple Runner in Godot 4
We'll create a beginner-friendly 3D endless runner where the player runs through an ancient jungle temple,
  ↪ avoids obstacles,
switches lanes, jumps, slides, and collects glowing orbs.

You don't need to know advanced programming or create models in Blender first. We'll build the first
  ↪ playable version using
Godot's built-in 3D shapes and GDScript.
1. What we'll build
Three-lane movement
Move left and right using the arrow keys or A and D.
Jump and dodge
Jump over low obstacles and avoid stone blocks.
Collect glowing orbs
Increase your score by collecting objects along the path.
Score and game over
The game gets faster as you run farther. Collisions end the run.
2. Download and open Godot
If you haven't installed it yet:
1.  Open the official Godot Windows download page.
2.  Download the standard Windows x86_64 version for most Windows 11 PCs.
3.  Extract the ZIP file and run the Godot executable.
4.  Open Godot's Project Manager and select Create New Project.
5.  Name it TempleRunner3D.
6.  Choose a folder for the project, select the Compatibility renderer if your PC has graphics
  ↪ compatibility issues, and create the project.
We'll use GDScript, which has a Python-like syntax and is included with Godot.
3. Create your first game scene
We'll create the entire first version using one script. This is easier for a beginner than creating lots of
  ↪  separate scenes and scripts.
In Godot:
1.  Click 3D Scene in the Scene dock.
2.  Select the root node named Node3D.
3.  Rename it to Main.
4.  Press Ctrl + S.
5.  Save the scene as main.tscn in your project folder.
6.  Select Main, click the Attach Script icon, and save the script as main.gd.
7.  Delete the default script contents and paste the code below.
4. Complete working game code
File: main.gd - Godot 4 - GDScript

extends Node3D

# TEMPLE RUNNER - BEGINNER VERSION
# Controls: A/D or arrows, Space to jump, R to restart.

var player: CharacterBody3D
var camera: Camera3D
var score_label: Label
var message_label: Label

var obstacles: Array[Node3D] = []
var orbs: Array[Node3D] = []
var pillars: Array[Node3D] = []

var lane := 1
var lanes := [-2.0, 0.0, 2.0]

var speed := 12.0
var gravity := 24.0
var jump_power := 9.0

var score := 0
var distance := 0.0
```

```
var spawn_timer := 0.0
var game_over := false


func _ready() -> void:
    randomize()
    make_world()
    make_player()
    make_camera()
    make_ui()
    spawn_timer = 1.0


func make_material(color: Color, glow := false) -> StandardMaterial3D:
    var mat := StandardMaterial3D.new()
    mat.albedo_color = color

    if glow:
        mat.emission_enabled = true
        mat.emission = color
        mat.emission_energy_multiplier = 2.0

    return mat


func make_box(
    parent: Node3D,
    size: Vector3,
    pos: Vector3,
    color: Color
) -> MeshInstance3D:
    var mesh_instance := MeshInstance3D.new()
    var mesh := BoxMesh.new()
    mesh.size = size
    mesh_instance.mesh = mesh
    mesh_instance.position = pos
    mesh_instance.material_override = make_material(color)
    parent.add_child(mesh_instance)
    return mesh_instance


func make_world() -> void:
    # Jungle temple atmosphere.
    var environment_node := WorldEnvironment.new()
    var environment := Environment.new()
    environment.background_mode = Environment.BG_SKY
    environment.ambient_light_source = Environment.AMBIENT_SOURCE_COLOR
    environment.ambient_light_color = Color(0.55, 0.65, 0.52)
    environment.ambient_light_energy = 0.8
    environment_node.environment = environment
    add_child(environment_node)

    var sun := DirectionalLight3D.new()
    sun.rotation_degrees = Vector3(-45, -25, 0)
    sun.light_energy = 1.3
    add_child(sun)

    # Main stone pathway.
    var floor_body := StaticBody3D.new()
    floor_body.name = "TemplePath"
    add_child(floor_body)

    make_box(
        floor_body,
        Vector3(7, 0.5, 120),
        Vector3(0, -0.25, -25),
        Color(0.36, 0.32, 0.26)
    )

    var floor_collision := CollisionShape3D.new()
    var floor_shape := BoxShape3D.new()
    floor_shape.size = Vector3(7, 0.5, 120)
    floor_collision.shape = floor_shape
    floor_collision.position = Vector3(0, -0.25, -25)
```

```
    floor_body.add_child(floor_collision)

    # Decorative path markings.
    for z in range(-80, 20, 5):
        make_box(
            self,
            Vector3(0.06, 0.02, 2),
            Vector3(0, 0.015, float(z)),
            Color(0.58, 0.49, 0.34)
        )

    # Temple pillars that move towards the player.
    for z in range(-55, 5, 8):
        make_pillar(-4.3, float(z))
        make_pillar(4.3, float(z))


func make_pillar(x: float, z: float) -> void:
    var pillar := Node3D.new()
    pillar.position = Vector3(x, 0, z)
    add_child(pillar)

    make_box(
        pillar,
        Vector3(0.8, 4, 0.8),
        Vector3(0, 2, 0),
        Color(0.48, 0.43, 0.34)
    )

    make_box(
        pillar,
        Vector3(1.2, 0.35, 1.2),
        Vector3(0, 4, 0),
        Color(0.64, 0.53, 0.35)
    )

    pillars.append(pillar)


func make_player() -> void:
    player = CharacterBody3D.new()
    player.name = "Player"
    player.position = Vector3(0, 0, 0)
    add_child(player)

    var capsule := CapsuleMesh.new()
    capsule.radius = 0.4
    capsule.height = 1.8

    var body_mesh := MeshInstance3D.new()
    body_mesh.mesh = capsule
    body_mesh.position.y = 0.9
    body_mesh.material_override = make_material(
        Color(0.15, 0.65, 1.0)
    )
    player.add_child(body_mesh)

    var collision := CollisionShape3D.new()
    var shape := CapsuleShape3D.new()
    shape.radius = 0.4
    shape.height = 1.8
    collision.shape = shape
    collision.position.y = 0.9
    player.add_child(collision)


func make_camera() -> void:
    camera = Camera3D.new()
    camera.position = Vector3(0, 5, 8)
    add_child(camera)
    camera.current = true
    camera.fov = 75
    camera.look_at(Vector3(0, 1, -8), Vector3.UP)
```

```
func make_ui() -> void:
    var canvas := CanvasLayer.new()
    add_child(canvas)

    score_label = Label.new()
    score_label.position = Vector2(20, 20)
    score_label.add_theme_font_size_override("font_size", 26)
    score_label.add_theme_color_override(
        "font_color", Color(1.0, 0.85, 0.3)
    )
    canvas.add_child(score_label)

    message_label = Label.new()
    message_label.anchor_left = 0.15
    message_label.anchor_top = 0.35
    message_label.anchor_right = 0.85
    message_label.anchor_bottom = 0.65
    message_label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    message_label.vertical_alignment = VERTICAL_ALIGNMENT_CENTER
    message_label.add_theme_font_size_override("font_size", 30)
    message_label.add_theme_color_override(
        "font_color", Color.WHITE
    )
    canvas.add_child(message_label)

    message_label.text = "TEMPLE RUNNER\nA/D or arrows to move\nSPACE to jump"


func spawn_obstacle() -> void:
    var obstacle := Area3D.new()
    obstacle.name = "StoneObstacle"

    var obstacle_mesh := BoxMesh.new()
    obstacle_mesh.size = Vector3(1.2, 1.2, 1.2)

    var visual := MeshInstance3D.new()
    visual.mesh = obstacle_mesh
    visual.position.y = 0.6
    visual.material_override = make_material(
        Color(0.65, 0.22, 0.12)
    )
    obstacle.add_child(visual)

    var collision := CollisionShape3D.new()
    var shape := BoxShape3D.new()
    shape.size = Vector3(1.2, 1.2, 1.2)
    collision.shape = shape
    collision.position.y = 0.6
    obstacle.add_child(collision)

    var chosen_lane := randi_range(0, 2)
    obstacle.position = Vector3(
        lanes[chosen_lane], 0, -40
    )
    add_child(obstacle)
    obstacles.append(obstacle)


func spawn_orb() -> void:
    var orb := Area3D.new()
    orb.name = "GoldenOrb"

    var orb_mesh := SphereMesh.new()
    orb_mesh.radius = 0.38
    orb_mesh.height = 0.76

    var visual := MeshInstance3D.new()
    visual.mesh = orb_mesh
    visual.material_override = make_material(
        Color(0.1, 0.95, 1.0), true
    )
    orb.add_child(visual)
```

```
    var collision := CollisionShape3D.new()
    var shape := SphereShape3D.new()
    shape.radius = 0.65
    collision.shape = shape
    orb.add_child(collision)

    var chosen_lane := randi_range(0, 2)
    orb.position = Vector3(
        lanes[chosen_lane], 1.1, -40
    )
    add_child(orb)
    orbs.append(orb)


func _input(event: InputEvent) -> void:
    if game_over:
        if event.is_action_pressed("ui_accept") \
        or event.is_action_pressed("ui_cancel") \
        or (event is InputEventKey and event.pressed \
        and event.keycode == KEY_R):
            get_tree().reload_current_scene()
        return

    if event.is_action_pressed("ui_left"):
        lane = maxi(0, lane - 1)

    if event.is_action_pressed("ui_right"):
        lane = mini(2, lane + 1)

    if event.is_action_pressed("ui_accept"):
        if player.is_on_floor():
            player.velocity.y = jump_power


func _physics_process(delta: float) -> void:
    if game_over:
        return

    # Smoothly move between the three lanes.
    player.position.x = move_toward(
        player.position.x, lanes[lane], 10.0 * delta
    )

    # Jumping and gravity.
    if not player.is_on_floor():
        player.velocity.y -= gravity * delta
    elif player.velocity.y < 0:
        player.velocity.y = 0

    player.move_and_slide()

    # Increase running speed and distance score.
    speed = min(speed + 0.12 * delta, 24.0)
    distance += speed * delta
    spawn_timer -= delta

    if spawn_timer <= 0:
        spawn_obstacle()

        if randf() < 0.8:
            spawn_orb()

        spawn_timer = randf_range(1.0, 1.6)

    # Move obstacles towards the player.
    for obstacle in obstacles.duplicate():
        obstacle.position.z += speed * delta

        if obstacle.position.z > 3:
            obstacles.erase(obstacle)
            obstacle.queue_free()
            continue

        var close_x := abs(
```

```
            obstacle.position.x - player.position.x
        ) < 0.85
        var close_z := abs(
            obstacle.position.z - player.position.z
        ) < 0.8

        # Jump over the obstacle to avoid a collision.
        if close_x and close_z and player.position.y < 0.85:
            end_game()

    # Move collectible orbs.
    for orb in orbs.duplicate():
        orb.position.z += speed * delta
        orb.rotate_y(3.0 * delta)

        if orb.position.z > 3:
            orbs.erase(orb)
            orb.queue_free()
            continue

        if orb.global_position.distance_to(
            player.global_position + Vector3(0, 1, 0)
        ) < 1.0:
            score += 10
            orbs.erase(orb)
            orb.queue_free()

    # Move the decorative pillars to simulate forward running.
    for pillar in pillars:
        pillar.position.z += speed * delta

        if pillar.position.z > 10:
            pillar.position.z -= 64

    score_label.text = (
        "ORBS: " + str(score)
        + "     DISTANCE: " + str(int(distance))
    )


func end_game() -> void:
    game_over = true
    message_label.text = (
        "GAME OVER!\n"
        + "Orbs: " + str(score)
        + "\nDistance: " + str(int(distance))
        + "\nPress R to restart"
    )
This prototype uses Godot's CharacterBody3D, 3D meshes, and collision shapes. These are standard Godot
  ↪ systems for character
movement and collision detection.

GitHub
+1
5. Run your game
Save your script: Press Ctrl + S.
Run the project: Press F6 to run the current scene, or F5 to run the project.
If Godot asks for the main scene: Choose main.tscn.
6. Controls
Key Action
<- or A Move one lane left
-> or D Move one lane right
Space or Enter  Jump
R   Restart after game over
7. What should you see?

When you run the game, you should see:
-   A stone temple path with pillars.
-   A blue capsule representing your player.
-   Red-brown stone obstacles approaching you.
-   Glowing cyan orbs that increase your score by 10.
-   A distance counter and increasing running speed.
-   A game-over message when you hit an obstacle without jumping.
```

```
The capsule is a temporary player model. Later, you can replace it with a proper animated character made in
  ↪ Blender.
8. If you get an error
-   Parser error: Check that you copied the entire script and kept the indentation.
-   Game opens but the player doesn't move: Click inside the game window first, then press the movement
  ↪ keys.
-   Black screen: Check that main.tscn is the main scene and that the script is attached to the root Main
  ↪ node.
-   Player falls through the floor: Check that the floor has its CollisionShape3D child and that the
  ↪ collision shape matches the floor dimensions.
9. What we can add next

Level 2 --- Colour gates
Add red, blue, and green gates with different gameplay effects.

Level 3 --- Real 3D character
Import a character from Blender and add running and jumping animations.

Level 4 --- Game polish
Add sound effects, a start menu, better graphics, and a restart button.
Start by running this first version. Once it works, we can improve the graphics and gameplay without making
  ↪ the project too complicated.
```

## A.3 Interaction 3: Boolean Type-Inference Error

### User prompt

```
Error at (320, 24): Cannot infer the type of "close_x" variable because the value doesn't have a set type.
```

### AI response

```
This error happens because Godot 4 cannot automatically determine the type of close_x. You can fix it by
  ↪ explicitly declaring
the variable as a Boolean (bool).
Fix the code
In main.gd, find this section inside _physics_process(delta):
var close_x := abs(
    obstacle.position.x - player.position.x
) < 0.85
var close_z := abs(
    obstacle.position.z - player.position.z
) < 0.8
Replace it with:
var close_x: bool = abs(
    obstacle.position.x - player.position.x
) < 0.85

var close_z: bool = abs(
    obstacle.position.z - player.position.z
) < 0.8
Then run the game
1.  Press Ctrl + S to save.
2.  Press F6 to run the current scene.
3.  Check whether the error disappears.
Why this works: close_x and close_z store true or false, depending on whether the player and obstacle are
  ↪ close enough.
Explicitly writing : bool tells Godot their types.
If another error appears, send me the exact error message and line number, and I'll help you fix it.
```

## A.4 Interaction 4: Indentation Error

### User prompt

```
error at (329,1):unexpected "Intedent" in class body
```

### AI response

```
This error means that the indentation on line 329 is incorrect. In Godot, code inside a function must be
  ↪ indented consistently.
Fix line 329
Find this section in your main.gd file:
if close_x and close_z and player.position.y < 0.85:
    end_game()
Make sure it is indented like this, inside the obstacle loop:
    for obstacle in obstacles.duplicate():
        obstacle.position.z += speed * delta

        if obstacle.position.z > 3:
            obstacles.erase(obstacle)
            obstacle.queue_free()
            continue

        var close_x: bool = abs(
            obstacle.position.x - player.position.x
        ) < 0.85

        var close_z: bool = abs(
            obstacle.position.z - player.position.z
        ) < 0.8

        if close_x and close_z and player.position.y < 0.85:
            end_game()
Important: The if close_x statement must be indented inside the for obstacle loop, not aligned with func
  ↪ _physics_process(delta: float) -> void:.
Try again
1.  Press Ctrl + S.
2.  Press F6 to run the scene.
If the error remains, send me a screenshot of lines 315--335 in Godot's script editor. I'll identify
  ↪ exactly where the indentation is wrong.
```

## A.5 Interaction 5: Request for Corrected Full Script

### User prompt

```
now give me " extends Node3D # TEMPLE RUNNER - BEGINNER VERSION # Controls: A/D or arrows, Space to jump, R
  ↪  to restart. var
player: CharacterBody3D var camera: Camera3D var score_label: Label var message_label: Label var obstacles:
  ↪  Array[Node3D] =
[] var orbs: Array[Node3D] = [] var pillars: Array[Node3D] = [] var lane := 1 var lanes := [-2.0, 0.0, 2.0]
  ↪  var speed := 12.0
var gravity := 24.0 var jump_power := 9.0 var score := 0 var distance := 0.0 var spawn_timer := 0.0 var
  ↪ game_over := false
func _ready() -> void: randomize() make_world() make_player() make_camera() make_ui() spawn_timer = 1.0
  ↪ func
make_material(color: Color, glow := false) -> StandardMaterial3D: var mat := StandardMaterial3D.new() mat.
  ↪ albedo_color =
color if glow: mat.emission_enabled = true mat.emission = color mat.emission_energy_multiplier = 2.0 return
  ↪  mat func
make_box( parent: Node3D, size: Vector3, pos: Vector3, color: Color ) -> MeshInstance3D: var mesh_instance
  ↪ :=
MeshInstance3D.new() var mesh := BoxMesh.new() mesh.size = size mesh_instance.mesh = mesh mesh_instance.
  ↪ position = pos
mesh_instance.material_override = make_material(color) parent.add_child(mesh_instance) return mesh_instance
  ↪  func make_world()
-> void: # Jungle temple atmosphere. var environment_node := WorldEnvironment.new() var environment :=
  ↪ Environment.new()
environment.background_mode = Environment.BG_SKY environment.ambient_light_source = Environment.
  ↪ AMBIENT_SOURCE_COLOR
environment.ambient_light_color = Color(0.55, 0.65, 0.52) environment.ambient_light_energy = 0.8
  ↪ environment_node.environment
= environment add_child(environment_node) var sun := DirectionalLight3D.new() sun.rotation_degrees =
  ↪ Vector3(-45, -25, 0)
sun.light_energy = 1.3 add_child(sun) # Main stone pathway. var floor_body := StaticBody3D.new() floor_body
  ↪ .name =
"TemplePath" add_child(floor_body) make_box( floor_body, Vector3(7, 0.5, 120), Vector3(0, -0.25, -25),
  ↪ Color(0.36, 0.32,
0.26) ) var floor_collision := CollisionShape3D.new() var floor_shape := BoxShape3D.new() floor_shape.size
  ↪ = Vector3(7, 0.5,
120) floor_collision.shape = floor_shape floor_collision.position = Vector3(0, -0.25, -25)
floor_body.add_child(floor_collision) # Decorative path markings. for z in range(-80, 20, 5): make_box(
  ↪ self, Vector3(0.06,
0.02, 2), Vector3(0, 0.015, float(z)), Color(0.58, 0.49, 0.34) ) # Temple pillars that move towards the
  ↪ player. for z in
range(-55, 5, 8): make_pillar(-4.3, float(z)) make_pillar(4.3, float(z)) func make_pillar(x: float, z:
  ↪ float) -> void: var
pillar := Node3D.new() pillar.position = Vector3(x, 0, z) add_child(pillar) make_box( pillar, Vector3(0.8,
  ↪ 4, 0.8),
Vector3(0, 2, 0), Color(0.48, 0.43, 0.34) ) make_box( pillar, Vector3(1.2, 0.35, 1.2), Vector3(0, 4, 0),
  ↪ Color(0.64, 0.53,
0.35) ) pillars.append(pillar) func make_player() -> void: player = CharacterBody3D.new() player.name = "
  ↪ Player"
player.position = Vector3(0, 0, 0) add_child(player) var capsule := CapsuleMesh.new() capsule.radius = 0.4
  ↪ capsule.height =
1.8 var body_mesh := MeshInstance3D.new() body_mesh.mesh = capsule body_mesh.position.y = 0.9 body_mesh.
  ↪ material_override =
make_material( Color(0.15, 0.65, 1.0) ) player.add_child(body_mesh) var collision := CollisionShape3D.new()
  ↪  var shape :=
CapsuleShape3D.new() shape.radius = 0.4 shape.height = 1.8 collision.shape = shape collision.position.y =
  ↪ 0.9
player.add_child(collision) func make_camera() -> void: camera = Camera3D.new() camera.position = Vector3
  ↪ (0, 5, 8)
add_child(camera) camera.current = true camera.fov = 75 camera.look_at(Vector3(0, 1, -8), Vector3.UP) func
  ↪ make_ui() -> void:
var canvas := CanvasLayer.new() add_child(canvas) score_label = Label.new() score_label.position = Vector2
  ↪ (20, 20)
score_label.add_theme_font_size_override("font_size", 26) score_label.add_theme_color_override( "font_color
  ↪ ", Color(1.0,
0.85, 0.3) ) canvas.add_child(score_label) message_label = Label.new() message_label.anchor_left = 0.15
message_label.anchor_top = 0.35 message_label.anchor_right = 0.85 message_label.anchor_bottom = 0.65
message_label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER message_label.vertical_alignment =
  ↪ VERTICAL_ALIGNMENT_CENTER
message_label.add_theme_font_size_override("font_size", 30) message_label.add_theme_color_override( "
  ↪ font_color", Color.WHITE
```

```
) canvas.add_child(message_label) message_label.text = "TEMPLE RUNNER\nA/D or arrows to move\nSPACE to jump
  ↪ " func
spawn_obstacle() -> void: var obstacle := Area3D.new() obstacle.name = "StoneObstacle" var obstacle_mesh :=
  ↪  BoxMesh.new()
obstacle_mesh.size = Vector3(1.2, 1.2, 1.2) var visual := MeshInstance3D.new() visual.mesh = obstacle_mesh
  ↪ visual.position.y
= 0.6 visual.material_override = make_material( Color(0.65, 0.22, 0.12) ) obstacle.add_child(visual) var
  ↪ collision :=
CollisionShape3D.new() var shape := BoxShape3D.new() shape.size = Vector3(1.2, 1.2, 1.2) collision.shape =
  ↪ shape
collision.position.y = 0.6 obstacle.add_child(collision) var chosen_lane := randi_range(0, 2) obstacle.
  ↪ position = Vector3(
lanes[chosen_lane], 0, -40 ) add_child(obstacle) obstacles.append(obstacle) func spawn_orb() -> void: var
  ↪ orb := Area3D.new()
orb.name = "GoldenOrb" var orb_mesh := SphereMesh.new() orb_mesh.radius = 0.38 orb_mesh.height = 0.76 var
  ↪ visual :=
MeshInstance3D.new() visual.mesh = orb_mesh visual.material_override = make_material( Color(0.1, 0.95, 1.0)
  ↪ , true )
orb.add_child(visual) var collision := CollisionShape3D.new() var shape := SphereShape3D.new() shape.radius
  ↪  = 0.65
collision.shape = shape orb.add_child(collision) var chosen_lane := randi_range(0, 2) orb.position =
  ↪ Vector3(
lanes[chosen_lane], 1.1, -40 ) add_child(orb) orbs.append(orb) func _input(event: InputEvent) -> void: if
  ↪ game_over: if
event.is_action_pressed("ui_accept") \ or event.is_action_pressed("ui_cancel") \ or (event is InputEventKey
  ↪  and event.pressed
\ and event.keycode == KEY_R): get_tree().reload_current_scene() return if event.is_action_pressed("ui_left
  ↪ "): lane = maxi(0,
lane - 1) if event.is_action_pressed("ui_right"): lane = mini(2, lane + 1) if event.is_action_pressed("
  ↪ ui_accept"): if
player.is_on_floor(): player.velocity.y = jump_power func _physics_process(delta: float) -> void: if
  ↪ game_over: return #
Smoothly move between the three lanes. player.position.x = move_toward( player.position.x, lanes[lane],
  ↪ 10.0 * delta ) #
Jumping and gravity. if not player.is_on_floor(): player.velocity.y -= gravity * delta elif player.velocity
  ↪ .y < 0:
player.velocity.y = 0 player.move_and_slide() # Increase running speed and distance score. speed = min(
  ↪ speed + 0.12 * delta,
24.0) distance += speed * delta spawn_timer -= delta if spawn_timer <= 0: spawn_obstacle() if randf() <
  ↪ 0.8: spawn_orb()
spawn_timer = randf_range(1.0, 1.6) # Move obstacles towards the player. for obstacle in obstacles.
  ↪ duplicate():
obstacle.position.z += speed * delta if obstacle.position.z > 3: obstacles.erase(obstacle) obstacle.
  ↪ queue_free() continue var
close_x: bool = abs( obstacle.position.x - player.position.x ) < 0.85 var close_z: bool = abs( obstacle.
  ↪ position.z -
player.position.z ) < 0.8 # Jump over the obstacle to avoid a collision. if close_x and close_z and player.
  ↪ position.y < 0.85:
end_game() # Move collectible orbs. for orb in orbs.duplicate(): orb.position.z += speed * delta orb.
  ↪ rotate_y(3.0 * delta) if
orb.position.z > 3: orbs.erase(orb) orb.queue_free() continue if orb.global_position.distance_to( player.
  ↪ global_position +
Vector3(0, 1, 0) ) < 1.0: score += 10 orbs.erase(orb) orb.queue_free() # Move the decorative pillars to
  ↪ simulate forward
running. for pillar in pillars: pillar.position.z += speed * delta if pillar.position.z > 10: pillar.
  ↪ position.z -= 64
score_label.text = ( "ORBS: " + str(score) + " DISTANCE: " + str(int(distance)) ) func end_game() -> void:
  ↪ game_over = true
message_label.text = ( "GAME OVER!\n" + "Orbs: " + str(score) + "\nDistance: " + str(int(distance)) + "\
  ↪ nPress R to restart"
) "- with updated code including " for obstacle in obstacles.duplicate(): obstacle.position.z += speed *
  ↪ delta if
obstacle.position.z > 3: obstacles.erase(obstacle) obstacle.queue_free() continue var close_x: bool = abs(
obstacle.position.x - player.position.x ) < 0.85 var close_z: bool = abs( obstacle.position.z - player.
  ↪ position.z ) < 0.8 if
close_x and close_z and player.position.y < 0.85: end_game()"
```

## AI response

```
Here is your complete updated main.gd code with the corrected close_x and close_z variables, proper
  ↪ indentation, and the
obstacle collision logic placed inside the loop.
Replace your entire existing main.gd script with this code.
```

```
extends Node3D

# TEMPLE RUNNER - BEGINNER VERSION
# Controls: A/D or arrows, Space to jump, R to restart.

var player: CharacterBody3D
var camera: Camera3D
var score_label: Label
var message_label: Label

var obstacles: Array[Node3D] = []
var orbs: Array[Node3D] = []
var pillars: Array[Node3D] = []

var lane := 1
var lanes := [-2.0, 0.0, 2.0]

var speed := 12.0
var gravity := 24.0
var jump_power := 9.0

var score := 0
var distance := 0.0
var spawn_timer := 0.0
var game_over := false


func _ready() -> void:
    randomize()
    make_world()
    make_player()
    make_camera()
    make_ui()
    spawn_timer = 1.0


func make_material(color: Color, glow: bool = false) -> StandardMaterial3D:
    var mat := StandardMaterial3D.new()
    mat.albedo_color = color

    if glow:
        mat.emission_enabled = true
        mat.emission = color
        mat.emission_energy_multiplier = 2.0

    return mat


func make_box(
    parent: Node3D,
    size: Vector3,
    pos: Vector3,
    color: Color
) -> MeshInstance3D:
    var mesh_instance := MeshInstance3D.new()
    var mesh := BoxMesh.new()
    mesh.size = size
    mesh_instance.mesh = mesh
    mesh_instance.position = pos
    mesh_instance.material_override = make_material(color)
    parent.add_child(mesh_instance)
    return mesh_instance


func make_world() -> void:
    # Jungle temple atmosphere.
    var environment_node := WorldEnvironment.new()
    var environment := Environment.new()
    environment.background_mode = Environment.BG_SKY
    environment.ambient_light_source = Environment.AMBIENT_SOURCE_COLOR
    environment.ambient_light_color = Color(0.55, 0.65, 0.52)
    environment.ambient_light_energy = 0.8
    environment_node.environment = environment
```

```
    add_child(environment_node)

    var sun := DirectionalLight3D.new()
    sun.rotation_degrees = Vector3(-45, -25, 0)
    sun.light_energy = 1.3
    add_child(sun)

    # Main stone pathway.
    var floor_body := StaticBody3D.new()
    floor_body.name = "TemplePath"
    add_child(floor_body)

    make_box(
        floor_body,
        Vector3(7, 0.5, 120),
        Vector3(0, -0.25, -25),
        Color(0.36, 0.32, 0.26)
    )

    var floor_collision := CollisionShape3D.new()
    var floor_shape := BoxShape3D.new()
    floor_shape.size = Vector3(7, 0.5, 120)
    floor_collision.shape = floor_shape
    floor_collision.position = Vector3(0, -0.25, -25)
    floor_body.add_child(floor_collision)

    # Decorative path markings.
    for z in range(-80, 20, 5):
        make_box(
            self,
            Vector3(0.06, 0.02, 2),
            Vector3(0, 0.015, float(z)),
            Color(0.58, 0.49, 0.34)
        )

    # Temple pillars.
    for z in range(-55, 5, 8):
        make_pillar(-4.3, float(z))
        make_pillar(4.3, float(z))


func make_pillar(x: float, z: float) -> void:
    var pillar := Node3D.new()
    pillar.position = Vector3(x, 0, z)
    add_child(pillar)

    make_box(
        pillar,
        Vector3(0.8, 4, 0.8),
        Vector3(0, 2, 0),
        Color(0.48, 0.43, 0.34)
    )

    make_box(
        pillar,
        Vector3(1.2, 0.35, 1.2),
        Vector3(0, 4, 0),
        Color(0.64, 0.53, 0.35)
    )

    pillars.append(pillar)


func make_player() -> void:
    player = CharacterBody3D.new()
    player.name = "Player"
    player.position = Vector3(0, 0, 0)
    add_child(player)

    var capsule := CapsuleMesh.new()
    capsule.radius = 0.4
    capsule.height = 1.8

    var body_mesh := MeshInstance3D.new()
```

```
    body_mesh.mesh = capsule
    body_mesh.position.y = 0.9
    body_mesh.material_override = make_material(
        Color(0.15, 0.65, 1.0)
    )
    player.add_child(body_mesh)

    var collision := CollisionShape3D.new()
    var shape := CapsuleShape3D.new()
    shape.radius = 0.4
    shape.height = 1.8
    collision.shape = shape
    collision.position.y = 0.9
    player.add_child(collision)


func make_camera() -> void:
    camera = Camera3D.new()
    camera.position = Vector3(0, 5, 8)
    add_child(camera)
    camera.current = true
    camera.fov = 75
    camera.look_at(Vector3(0, 1, -8), Vector3.UP)


func make_ui() -> void:
    var canvas := CanvasLayer.new()
    add_child(canvas)

    score_label = Label.new()
    score_label.position = Vector2(20, 20)
    score_label.add_theme_font_size_override("font_size", 26)
    score_label.add_theme_color_override(
        "font_color", Color(1.0, 0.85, 0.3)
    )
    canvas.add_child(score_label)

    message_label = Label.new()
    message_label.anchor_left = 0.15
    message_label.anchor_top = 0.35
    message_label.anchor_right = 0.85
    message_label.anchor_bottom = 0.65
    message_label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    message_label.vertical_alignment = VERTICAL_ALIGNMENT_CENTER
    message_label.add_theme_font_size_override("font_size", 30)
    message_label.add_theme_color_override(
        "font_color", Color.WHITE
    )
    canvas.add_child(message_label)

    message_label.text = "TEMPLE RUNNER\nA/D or arrows to move\nSPACE to jump"


func spawn_obstacle() -> void:
    var obstacle := Area3D.new()
    obstacle.name = "StoneObstacle"

    var obstacle_mesh := BoxMesh.new()
    obstacle_mesh.size = Vector3(1.2, 1.2, 1.2)

    var visual := MeshInstance3D.new()
    visual.mesh = obstacle_mesh
    visual.position.y = 0.6
    visual.material_override = make_material(
        Color(0.65, 0.22, 0.12)
    )
    obstacle.add_child(visual)

    var collision := CollisionShape3D.new()
    var shape := BoxShape3D.new()
    shape.size = Vector3(1.2, 1.2, 1.2)
    collision.shape = shape
    collision.position.y = 0.6
    obstacle.add_child(collision)
```

```
    var chosen_lane := randi_range(0, 2)
    obstacle.position = Vector3(
        lanes[chosen_lane], 0, -40
    )
    add_child(obstacle)
    obstacles.append(obstacle)


func spawn_orb() -> void:
    var orb := Area3D.new()
    orb.name = "GoldenOrb"

    var orb_mesh := SphereMesh.new()
    orb_mesh.radius = 0.38
    orb_mesh.height = 0.76

    var visual := MeshInstance3D.new()
    visual.mesh = orb_mesh
    visual.material_override = make_material(
        Color(0.1, 0.95, 1.0), true
    )
    orb.add_child(visual)

    var collision := CollisionShape3D.new()
    var shape := SphereShape3D.new()
    shape.radius = 0.65
    collision.shape = shape
    orb.add_child(collision)

    var chosen_lane := randi_range(0, 2)
    orb.position = Vector3(
        lanes[chosen_lane], 1.1, -40
    )
    add_child(orb)
    orbs.append(orb)


func _input(event: InputEvent) -> void:
    if game_over:
        if event.is_action_pressed("ui_accept") \
        or event.is_action_pressed("ui_cancel") \
        or (event is InputEventKey and event.pressed \
        and event.keycode == KEY_R):
            get_tree().reload_current_scene()
        return

    if event.is_action_pressed("ui_left"):
        lane = maxi(0, lane - 1)

    if event.is_action_pressed("ui_right"):
        lane = mini(2, lane + 1)

    if event.is_action_pressed("ui_accept"):
        if player.is_on_floor():
            player.velocity.y = jump_power


func _physics_process(delta: float) -> void:
    if game_over:
        return

    # Smoothly move between lanes.
    player.position.x = move_toward(
        player.position.x, lanes[lane], 10.0 * delta
    )

    # Jumping and gravity.
    if not player.is_on_floor():
        player.velocity.y -= gravity * delta
    elif player.velocity.y < 0:
        player.velocity.y = 0

    player.move_and_slide()
```

```
    # Increase speed and distance.
    speed = min(speed + 0.12 * delta, 24.0)
    distance += speed * delta
    spawn_timer -= delta

    if spawn_timer <= 0:
        spawn_obstacle()

        if randf() < 0.8:
            spawn_orb()

        spawn_timer = randf_range(1.0, 1.6)

    # Move obstacles towards the player.
    for obstacle in obstacles.duplicate():
        obstacle.position.z += speed * delta

        if obstacle.position.z > 3:
            obstacles.erase(obstacle)
            obstacle.queue_free()
            continue

        var close_x: bool = abs(
            obstacle.position.x - player.position.x
        ) < 0.85

        var close_z: bool = abs(
            obstacle.position.z - player.position.z
        ) < 0.8

        # Collision detection.
        if close_x and close_z and player.position.y < 0.85:
            end_game()

    # Move collectible orbs.
    for orb in orbs.duplicate():
        orb.position.z += speed * delta
        orb.rotate_y(3.0 * delta)

        if orb.position.z > 3:
            orbs.erase(orb)
            orb.queue_free()
            continue

        if orb.global_position.distance_to(
            player.global_position + Vector3(0, 1, 0)
        ) < 1.0:
            score += 10
            orbs.erase(orb)
            orb.queue_free()

    # Move decorative pillars.
    for pillar in pillars:
        pillar.position.z += speed * delta

        if pillar.position.z > 10:
            pillar.position.z -= 64

    score_label.text = (
        "ORBS: " + str(score)
        + "     DISTANCE: " + str(int(distance))
    )


func end_game() -> void:
    game_over = true
    message_label.text = (
        "GAME OVER!\n"
        + "Orbs: " + str(score)
        + "\nDistance: " + str(int(distance))
        + "\nPress R to restart"
    )
Run the updated game
```

```
1.  Replace all the old code with the code above.
2.  Press Ctrl + S.
3.  Check the script editor for red error indicators.
4.  Press F6 to run main.tscn. If Godot asks for a main scene, select this scene.
Important: This fixes the indentation and type-declaration errors you reported. If a new error appears,
  ↪ send me its exact
message and line number so we can fix it.
```

## A.6 Interaction 6: Adding Blender Scenery

### User prompt

```
i did all the thing and game is running now add scenario like tree , colorful gate in blender and show me how to add this step by step
```

### AI response

```
Upgrade your Temple Runner: Jungle Trees + Colourful Gates
Great! Since your Temple Runner is already running in Godot 4, let's make it look like a real 3D endless-runner game by
creating scenery in Blender 5.2 and importing it into Godot.

We'll work in two programs:
-   Blender: Create 3D trees and colourful gates.
-   Godot 4: Place the models beside your running path, animate the scenery, and add collision or gameplay effects to the gates.
What we'll create

1. Low-poly jungle trees
Green leafy canopies, brown trunks, and simple geometry that keeps the game lightweight. We'll place them along both sides of the path.

2. Colourful gates
Three-dimensional gates with bright red, blue, and green frames. The gates can later become obstacles or colour-theory challenges.

3. A complete game environment
Combine the trees, gates, stone pathway, and existing obstacles into one cohesive scene.
Part 1 --- Create a jungle tree in Blender 5.2
Goal: A low-poly tree approximately 5 metres tall, suitable for your game.

Step 1. Create the trunk
1.  Open Blender and start a new General project.
2.  Select the default cube and press X, then Enter to delete it.
3.  Press Shift + A -> Mesh -> Cylinder.
4.  With the cylinder selected, use these shortcuts:
Shortcut    Action
S, Z, 2, Enter  Make it taller
S, Shift + Z, 0.4, Enter    Make it narrower horizontally
G, Z, 2, Enter  Raise it above the ground
This creates the brown tree trunk.
Step 2. Add green leaves
1.  Press Shift + A -> Mesh -> Icosphere.
2.  Press G, Z, 3.2, Enter to position the leaves above the trunk.
3.  Press S, 1.5, Enter to enlarge them.
4.  Repeat twice to create two more foliage clusters. Position one higher and one slightly to the side.
Your tree should now have one trunk and three leafy clusters.
Step 3. Add colours
Select the trunk and open Material Properties (the red checker/sphere material icon).
1.  Click New.
2.  Set the Base Color to brown.
3.  Select the leaf objects one at a time.
4.  Give them green materials, using different shades such as dark green, leaf green, and lime green.
For a stylized look, right-click the leaves and select Shade Flat.
Step 4. Save and export the tree
1.  Select the trunk and all three leaf objects.
2.  Press Ctrl + J to join them into one object. Their assigned materials should be retained.
3.  Rename the object JungleTree.
4.  Press Ctrl + Shift + S and save the Blender project as JungleTree.blend.
Now export the model:
-   Open File -> Export -> glTF 2.0.
-   Choose glTF Binary (.glb).
-   Save as JungleTree.glb.
The .glb format packages the model and supported materials together, making it convenient for Godot.

Blender 5.2 LTS Manual
+1
Part 2 --- Create a colourful gate in Blender

We'll create a rectangular frame with three different colours. Later, you can turn it into a colour-theory challenge.
```

```
Step 1. Create the two vertical posts
1.  In Blender, create a new General project.
2.  Delete the default cube.
3.  Press Shift + A -> Mesh -> Cube.
4.  Use these shortcuts:
Shortcut    Action
S, X, 0.2, Enter    Make the post narrow
S, Y, 0.2, Enter    Make it narrow in depth
S, Z, 7.5, Enter    Make it tall
G, Z, 1.5, Enter    Raise it
This makes a tall post. Scale it down if needed; the final gate should be about 3 metres tall.
5. Duplicate the post with Shift + D, then X, then 4, then Enter to create the other side. Adjust the
  ↪ separation so the frame is around 4 metres wide.
Step 2. Add the top bar
1.  Add another cube.
2.  Scale it into a horizontal beam.
3.  Place it above the two posts, connecting them.
You now have a basic rectangular gate frame.
Step 3. Add colourful materials
Create three materials and assign them to the frame sections:
-   Red (#FF3030)
-   Blue (#168BFF)
-   Green (#36FF75)
For a glowing effect, select a material and open Material Properties. Use an emission shader or the
  ↪ Principled BSDF emission
settings available in your Blender version. The emissive appearance may look different in Blender and Godot
  ↪ , so test it after
importing.
Step 4. Export the gate
1.  Select all gate pieces.
2.  Press Ctrl + J to join them into one object while preserving the material assignments.
3.  Rename it ColourGate.
4.  Save the project as ColourGate.blend.
5.  Export through File -> Export -> glTF 2.0 -> glTF Binary (.glb).
6.  Name the exported file ColourGate.glb.
Part 3 --- Import the models into Godot 4
Godot supports importing .glb files directly as 3D scenes.

Godot Engine (stable) documentation in English

Step 1. Create an assets folder
In Godot's FileSystem dock, right-click res:// and create a folder named assets.

Step 2. Copy your models
Copy both files into the new folder:
-   JungleTree.glb
-   ColourGate.glb
Godot will import them automatically. Wait for the import process to finish.

Step 3. Test the imported models
Drag each .glb file from the FileSystem dock into your open 3D scene. Check that its scale and materials
  ↪ look correct.
You can remove the temporary copies after testing.
Part 4 --- Add trees and gates to your existing game using code
Your current game creates the path, pillars, player, obstacles, and orbs from main.gd. We'll extend it to
  ↪ load your Blender models automatically.
Step 1. Add these variables near the top of main.gd
Place them below your existing pillars array:
var trees: Array[Node3D] = []
var gates: Array[Node3D] = []

const TREE_SCENE: PackedScene = preload(
    "res://assets/JungleTree.glb"
)

const GATE_SCENE: PackedScene = preload(
    "res://assets/ColourGate.glb"
)
Make sure the filenames and capitalization exactly match the files in your assets folder.
Step 2. Create the scenery
Add these two functions anywhere outside your other functions:
func make_trees() -> void:
    for i in range(12):
        for side in [-1.0, 1.0]:
```

```
            var tree := TREE_SCENE.instantiate() as Node3D
            add_child(tree)

            tree.position = Vector3(
                side * randf_range(5.0, 7.0),
                0.0,
                -float(i) * 10.0 - 10.0
            )

            tree.scale = Vector3.ONE * randf_range(0.8, 1.2)
            trees.append(tree)


func make_gates() -> void:
    for i in range(3):
        var gate := GATE_SCENE.instantiate() as Node3D
        add_child(gate)

        gate.position = Vector3(
            0.0,
            0.0,
            -40.0 - float(i) * 45.0
        )

        gates.append(gate)
This creates 24 trees along the sides of the pathway and three gates along the running route. The gates are
  ↪  decorative for now.
Step 3. Call the new functions
Find your existing _ready() function:
func _ready() -> void:
    randomize()
    make_world()
    make_player()
    make_camera()
    make_ui()
    spawn_timer = 1.0
Replace it with:
func _ready() -> void:
    randomize()
    make_world()
    make_trees()
    make_gates()
    make_player()
    make_camera()
    make_ui()
    spawn_timer = 1.0
Step 4. Make the scenery move with the runner
At the bottom of _physics_process(delta), after your existing pillar movement loop, add:
    # Move trees towards the player and recycle them.
    for tree in trees:
        tree.position.z += speed * delta

        if tree.position.z > 10.0:
            tree.position.z -= 120.0

    # Move gates towards the player and recycle them.
    for gate in gates:
        gate.position.z += speed * delta

        if gate.position.z > 10.0:
            gate.position.z -= 135.0
Keep this code inside _physics_process(), aligned with the other for loops in that function.
Press Ctrl + S, then F6 to test the scene.
Note: This first pass uses the gates as visual scenery, without collision or colour-changing gameplay. We'
  ↪ ll add those
behaviours after confirming the imported models work correctly.
Part 5 --- Check your result
-   The trees appear outside the stone path.
-   The red, blue, and green gate frame appears on the path.
-   The scenery moves toward the player.
-   Trees and gates recycle instead of disappearing permanently.
-   Your existing jumping, lane movement, score, and game-over logic still work.
If a tree appears too small or too large, adjust its scale in Blender or in the Godot code. If a gate is
  ↪ too wide, adjust its
```

```
horizontal dimensions in Blender.
My recommendation: First finish and import the tree and gate models. Once they're visible in your game, the
  ↪ next step is to
make the gates change colours dynamically and introduce colour-theory rules---for example, selecting a
  ↪ complementary colour
to pass through a gate safely.
```